ATIVIDADE AVALIATIVA – AULA 03

Modelagem no MongoDB: *Flexibilidade x Estrutura*

Banco de Dados Não Relacional · Prof.ª Lucineide · FATEC

- BD_SE – Sem Estrutura
- BD_CE – Com Estrutura
- Análise Crítica
- Raciocínio de Arquitetura

PARTE 1

Banco Sem Estrutura – BD_SE

1 Tarefa 1 – Inserção Livre

✓ Sucesso esperado

Criação do banco e inserção de três jogadores com estruturas propositalmente diferentes: apenas nome, nome com ranking e patrocinador, e nome com campo inesperado **redeSocial**.

MongoDB Shell — BD_SE · Criar banco e inserir jogadores

```
// Passo 1 - Selecionar / criar o banco
use BD_SE

// Passo 2 - Inserir jogadores com estruturas variadas
db.jogadores.insertMany([

  // Jogador 1: apenas o campo nome
  {
    nome: "Felipe"
  },

  // Jogador 2: nome, ranking e patrocinador
  {
    nome: "Marina",
    ranking: 1200,
    patrocinador: "HyperGear"
  },

  // Jogador 3: campo inesperado "redeSocial"
  {
    nome: "Rafael",
    nick: "RafaX",
    redeSocial: "@rafax_games"
  }

])
```

✓ Todos os documentos inseridos com sucesso — sem validação de schema ativa

2 Tarefa 2 – Análise Crítica

Questão discursiva

1 Todos os documentos possuem o mesmo padrão?

Não. Cada documento inserido possui uma estrutura diferente: o primeiro tem apenas **nome**; o segundo tem **nome**, **ranking** e **patrocinador**; o terceiro inclui o campo incomum **redeSocial**. No BD_SE, o MongoDB aceita qualquer estrutura sem questionar, pois não há um schema validator definido na coleção.

2 Isso pode gerar problemas em consultas futuras?

Sim. Quando uma aplicação tentar consultar todos os jogadores esperando determinados campos — como **ranking** ou **nick** — parte dos documentos retornará `null` ou simplesmente não trará o campo. Isso obriga o código da aplicação a tratar cada caso individualmente, aumentando a complexidade e a chance de erros silenciosos.

3 Cite um risco desse modelo em sistemas grandes.

Em sistemas de grande escala, a ausência de estrutura pode gerar **inconsistência de dados acumulada ao longo do tempo**: diferentes versões da aplicação inserem campos com nomes distintos para a mesma informação (ex.: `redeSocial` vs `rede_social` vs `instagram`), tornando impossível realizar agregações e relatórios confiáveis sem uma limpeza massiva dos dados. Além disso, a falta de tipagem obrigatória permite que um campo como `ranking` seja salvo ora como número, ora como string, quebrando ordenações e cálculos.

PARTE 2 Banco Com Estrutura – BD_CE

3 Tarefa 3 – Inserção Correta (válida)

✓ Deve funcionar

Primeiro cria-se o banco e a coleção com schema validator. Em seguida insere-se um documento que respeita todos os campos obrigatórios e os tipos BSON definidos.

MongoDB Shell — BD_CE · Criar banco, schema e inserção válida

```
// Passo 1 - Selecionar / criar o banco
use BD_CE

// Passo 2 - Criar coleção jogadores com schema validator
db.createCollection("jogadores", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "nick", "ranking"],
      properties: {
        nome: {
          bsonType: "string",
          description: "Nome deve ser string e é obrigatório"
        },
        nick: {
          bsonType: "string"
        },
        ranking: {
          bsonType: "int",
          minimum: 0
        },
        patrocinador: {
          bsonType: "string"
        }
      }
    }
  }
})

// Passo 3 - Inserção válida: todos os campos obrigatórios presentes e com
tipos corretos
db.jogadores.insertOne({
  nome: "Fernanda",
  nick: "FerGamer",
  ranking: NumberInt(980),
  patrocinador: "NovaTech"
})
```

✓ Documento inserido com sucesso — todos os campos obrigatórios presentes e tipados corretamente

4 Tarefa 4 – Inserção Inválida (deve gerar erro)

✗ Erro esperado

Dois exemplos de inserções inválidas são demonstrados: o primeiro sem o campo obrigatório **nick**, e o segundo com **ranking** enviado como string em vez de int.

MongoDB Shell — BD_CE · Inserção sem campo obrigatório

```
// Tentativa 1 - Falta o campo obrigatório "nick" e "ranking"
db.jogadores.insertOne({
  nome: "Bruno"
})

// MongoServerError: Document failed validation
// Reason: campo "nick" é obrigatório (required)
//         campo "ranking" é obrigatório (required)
```

MongoDB Shell — BD_CE · Inserção com tipo incorreto

```
// Tentativa 2 - ranking enviado como string em vez de int
db.jogadores.insertOne({
  nome: "Carla",
  nick: "CarlaX",
  ranking: "alto"  // ← tipo errado: string ao invés de int
})

// MongoServerError: Document failed validation
// Reason: campo "ranking" deve ser do tipo int, recebeu string
```

✗ Ambas as inserções foram rejeitadas pelo validator — comportamento correto do BD_CE

5 Tarefa 5 – Comparação Técnica

Questão discursiva

Critério	BD_SE (Sem Estrutura)	BD_CE (Com Estrutura)
Segurança para produção	Baixa — qualquer dado é aceito	**Alta** — validação garantida pelo MongoDB
Flexibilidade para prototipar	**Alta** — sem restrições de campo	Limitada — schema precisa ser definido antes
Consistência dos dados	Variável — depende da aplicação	**Garantida** pelo validator
Custo de manutenção	Alto — necessita validação na aplicação	Baixo — validação centralizada no banco

1. **Qual banco oferece mais segurança para produção?**

   O **BD_CE**. Com o schema validator ativo, o próprio MongoDB rejeita documentos malformados antes de persistir, independente da camada de aplicação. Isso elimina uma classe inteira de bugs causados por dados faltantes ou com tipos incorretos.

2. **Qual banco oferece mais flexibilidade para prototipação?**

   O **BD_SE**. Na fase inicial de um projeto, os campos e tipos ainda estão sendo descobertos. Não ter um schema obrigatório permite iterar rapidamente sem precisar alterar a estrutura da coleção a cada mudança de requisito.

3. **Em qual cenário você utilizaria cada modelo?**

   BD_SE é ideal para *provas de conceito*, projetos em fase de exploração, coleta de dados de origens externas variadas (logs, eventos de IoT) onde a estrutura não é controlável. Já o **BD_CE** é a escolha correta para sistemas em produção com múltiplos desenvolvedores, integrações com APIs, ou qualquer contexto onde a integridade dos dados impacta diretamente regras de negócio — como cadastro de usuários, transações financeiras e rankings competitivos.

PARTE 3 Raciocínio de Arquitetura

6 Situação-Problema – Empresa em crescimento com API pública

Arquitetura

Uma empresa de jogos está crescendo rapidamente e pretende integrar seu banco com uma API pública, com acesso simultâneo de milhares de usuários.

RECOMENDAÇÃO TÉCNICA

A recomendação é utilizar o **BD_CE** — banco com schema validator definido — como modelo para o ambiente de produção.

Justificativa 1 — Integridade em escala: com milhares de usuários simultâneos e múltiplos clientes consumindo a API (aplicações web, mobile, parceiros externos), é inevitável que diferentes versões do cliente enviem payloads levemente diferentes. Sem um schema no banco, erros de estrutura só seriam detectados na camada de leitura, gerando falhas difíceis de rastrear. O validator do BD_CE cria uma barreira de segurança na origem.

Justificativa 2 — Confiabilidade da API: uma API pública precisa garantir contratos de dados estáveis. Se o banco aceitar documentos sem os campos esperados, a API poderá retornar respostas incompletas para os consumidores externos, ferindo o contrato e gerando retrabalho. Com o BD_CE, qualquer dado que passa pelo banco já foi validado.

Justificativa 3 — Manutenção e colaboração: equipes grandes que trabalham em paralelo no mesmo sistema precisam de uma fonte única de verdade sobre a estrutura dos dados. O schema centralizado no BD_CE serve como documentação viva e evita que diferentes times interpretem os campos de formas distintas.

Conclusão: o BD_SE pode coexistir como ambiente de sandbox para experimentação, mas a base que sustenta a API pública e os dados de produção deve obrigatoriamente ser o **BD_CE**. Banco de dados não é lugar de improviso — estrutura é decisão estratégica.